integracion

notificaciones

NotificacionTransportAmqp
- private rabbitTemplate: RabbitTemplate
- public enviar(solicitud: SolicitudNotificacionMensaje): void
- public nombre(): String

«record» SolicitudNotificacionMensaje
- destinatario: String
- mensaje: String
- medio: String

NotificacionTransportRest
- private restClient: RestClient
- public enviar(solicitud: SolicitudNotificacionMensaje): void
- public nombre(): String

Se crea una sola implementación según la property notificaciones.transporte: rest (predeterminado) → POST /notificaciones (8084) amqp → exchange donatrack.notificaciones, routing key notificacion.enviar (RabbitMQConfig solo se carga con amqp)

config

RabbitMQConfig
- public EXCHANGE: String = "donatrack.notificaciones"
- public ROUTING_KEY: String = "notificacion.enviar"
- public QUEUE: String = "notificaciones.queue"
- public notificacionesExchange(): TopicExchange
- public notificacionesQueue(): Queue
- public notificacionesBinding(notificacionesQueue: Queue, notificacionesExchange: TopicExchange): Binding
- public jsonMessageConverter(): MessageConverter
- public rabbitTemplate(connectionFactory: ConnectionFactory, jsonMessageConverter: MessageConverter): RabbitTemplate

services

AsignacionBatchService
- private ESTADO_EN_DEPOSITO: String = "EN_DEPOSITO"
- private donacionesRepositorio: DonacionesRepositoryInterface
- private asignacionService: AsignacionDonacionesService
- private candidatosRepositorio: CandidatosAsignacionRepositoryInterface
- public ejecutarMatchmakingNocturno(): void «@Scheduled 0 30 3 * * *»
- private calcularRanking(donacion: Donacion): RankingPrecalculado

SegmentadorDonacionesService
- private donacionesRepository: DonacionesRepositoryInterface
- private donanteRepository: PersonaDonanteRepositoryInterface
- private incentivosClient: IncentivosClient
- public segmentar(carga: CargaDonacion): List<Donacion>
- public cargarDonaciones(donaciones: List<Donacion>): void
- private notificarIncentivos(idDonante: UUID, bienes: List<Bien>): void

DonacionService
- private repositorio: DonacionesRepositoryInterface
- private mapper: DonacionMapper
- private asignacionService: AsignacionDonacionesService
- private entidadesRepositorio: EntidadesBeneficiariasRepositoryInterface
- private entidadMapper: EntidadBeneficiariaMapper
- private donanteRepositorio: PersonaDonanteRepositoryInterface
- private notificacionClient: NotificacionClient
- private incentivosClient: IncentivosClient
- private validador: DonacionesValidator
- private entidadesValidator: EntidadesBeneficiariasValidator
- private candidatosRepositorio: CandidatosAsignacionRepositoryInterface
- public obtenerDonaciones(estado: String, idDonante: UUID, subcategoria: String): List<DonacionResponseDTO>
- public obtenerPorId(id: UUID): DonacionResponseDTO
- public cambiarEstado(id: UUID, dto: CambioEstadoRequestDTO): void
- public modificarBien(id: UUID, dto: BienRequestDTO): void
- public obtenerCandidatos(idDonacion: UUID): CandidatosAsignacionResponseDTO
- public obtenerCandidatosPendientes(): List<CandidatosPendientesResponseDTO>
- public asignar(idDonacion: UUID, dto: AsignacionRequestDTO): void
- public eliminar(id: UUID): void
- private mapearRanking(ranking: List<ResultadoAsignacion>): List<EntidadBeneficiariaResponseDTO>
- private notificarDonacionExitosa(donacion: Donacion): void
- private notificarAsignacion(donacion: Donacion, entidad: EntidadBeneficiaria): void

AsignacionDonacionesService
- private algoritmoSemantico: AlgoritmoCompatibilidadSemantica
- private algoritmoSubAtendidos: AlgoritmoPrioridadSubAtendidos
- private entidadesRepository: EntidadesBeneficiariasRepositoryInterface
- public generarRanking(donacion: Donacion): ResultadoMatchmaking
- private intersectar(a: List<ResultadoAsignacion>, b: List<ResultadoAsignacion>): List<ResultadoAsignacion>

AlgoritmoPrioridadSubAtendidos
- private MESES_TRIMESTRE: int = 3
- private donacionesRepository: DonacionesRepositoryInterface
- protected calcularPuntaje(donacion: Donacion, entidad: EntidadBeneficiaria): double

ResultadoMatchmaking
- private coincidencias: List<ResultadoAsignacion>
- private rankingSemantico: List<ResultadoAsignacion>
- private rankingSubAtendidos: List<ResultadoAsignacion>
- public huboCoincidencias(): boolean

AlgoritmoCompatibilidadSemantica
- protected calcularPuntaje(donacion: Donacion, entidad: EntidadBeneficiaria): double
- protected incluir(puntaje: double): boolean

«abstract» AlgoritmoAsignacionBase
- protected TOP_N: int = 10
- public final evaluar(donacion: Donacion, entidades: List<EntidadBeneficiaria>): List<ResultadoAsignacion>
- protected calcularPuntaje(donacion: Donacion, entidad: EntidadBeneficiaria): double
- protected incluir(puntaje: double): boolean

PersonaDonanteService
- private repositorio: PersonaDonanteRepositoryInterface
- private validador: PersonasValidator
- private mapper: PersonaDonanteMapper
- public registrar(dto: PersonaDonanteRequestDTO): PersonaDonanteResponseDTO
- public obtenerDonante(id: UUID): PersonaDonanteResponseDTO
- public obtenerDonantes(estado: String): List<PersonaDonanteResponseDTO>
- public cambiarEstado(id: UUID, dto: EstadoDonanteRequestDTO): void
- public modificarContacto(id: UUID, dto: MedioDeContactoRequestDTO): void
- public modificarRepresentante(id: UUID, dto: RepresentanteRequestDTO): void
- public actualizar(id: UUID, dto: PersonaDonanteUpdateRequestDTO): PersonaDonanteResponseDTO
- public eliminar(id: UUID): void

LogisticaEventosService
- private donacionesRepositorio: DonacionesRepositoryInterface
- private entidadesRepositorio: EntidadesBeneficiariasRepositoryInterface
- private donanteRepositorio: PersonaDonanteRepositoryInterface
- private notificacionClient: NotificacionClient
- private incentivosClient: IncentivosClient
- public procesarInicioRuta(dto: InicioRutaCallbackDTO): void
- public procesarEntregaExitosa(dto: EntregaExitosaCallbackDTO): void
- public procesarEntregaFallida(dto: EntregaFallidaCallbackDTO): void
- private notificarInicioRutaEntidad(donacion: Donacion, urlMapa: String): void
- private notificarInicioRutaDonante(donacion: Donacion, urlMapa: String): void
- private notificarEntregaExitosaEntidad(donacion: Donacion, comprobante: String): void
- private notificarEntregaExitosaDonante(donacion: Donacion, comprobante: String): void
- private enviarComprobante(dto: EntregaExitosaCallbackDTO): String
- private notificarEntregaFallidaEntidad(donacion: Donacion, motivo: String, reprogramable: boolean): void
- private notificarEntregaFallidaDonante(donacion: Donacion, motivo: String, reprogramable: boolean): void
- private notificarEntregaFallidaAdmin(donacion: Donacion, motivo: String, reprogramable: boolean): void
- private obtenerDonacionOFallar(id: UUID): Donacion
- private obtenerEmail(contactos: List<MedioDeContacto>): String

PlanificacionEntregasService
- private ESTADO_ASIGNADA: String = "ASIGNACION_REALIZADA"
- private donacionesRepositorio: DonacionesRepositoryInterface
- private entidadesRepositorio: EntidadesBeneficiariasRepositoryInterface
- private brokerClient: BrokerLogisticaClient
- private validador: DonacionesValidator
- public despacharYa(idDonaciones: List<UUID>, proveedor: String): EnvioLogisticaResponseDTO
- public despacharPendientesDeDia(): void «@Scheduled 0 0 4 * * *»
- private validarYObtener(idsDonaciones: List<UUID>): Map<UUID, Donacion>
- private tieneDireccionResoluble(donacion: Donacion): boolean
- private entidadDe(donacion: Donacion): EntidadBeneficiaria
- private aDonacionEnvio(donacion: Donacion): DonacionEnvio
- private llamarABroker(proveedor: String, pedido: List<DonacionEnvio>): ResultadoEnvio

EntidadesBeneficiariasService
- private repositorio: EntidadesBeneficiariasRepositoryInterface
- private validador: EntidadesBeneficiariasValidator
- private mapper: EntidadBeneficiariaMapper
- private dependenciasMapper: PersonaDonanteMapper
- public guardar(dto: EntidadBeneficiariaRequestDTO): EntidadBeneficiariaResponseDTO
- public obtenerTodas(): List<EntidadBeneficiariaResponseDTO>
- public obtenerPorId(id: UUID): EntidadBeneficiariaResponseDTO
- public actualizar(id: UUID, dtoNueva: EntidadBeneficiariaRequestDTO): void
- public eliminarEntidad(id: UUID): void
- public agregarCampaniaAEntidad(idEntidad: UUID, dto: CampaniaRequestDTO): CampaniaResponseDTO
- public agregarNecesidadACampania(entidadId: UUID, campaniaId: UUID, dto: NecesidadRequestDTO): NecesidadResponseDTO
- public obtenerNecesidadesDeCampania(entidadId: UUID, campaniaId: UUID): List<NecesidadResponseDTO>
- public obtenerNecesidad(entidadId: UUID, campaniaId: UUID, necesidadId: UUID): NecesidadResponseDTO
- public actualizarNecesidad(entidadId: UUID, campaniaId: UUID, necesidadId: UUID, dto: NecesidadRequestDTO): NecesidadResponseDTO
- public eliminarNecesidad(entidadId: UUID, campaniaId: UUID, necesidadId: UUID): void

InactividadDonantesService
- private DIAS_LIMITE_INACTIVIDAD: int = 20
- private donanteRepository: PersonaDonanteRepositoryInterface
- private notificacionClient: NotificacionClient
- public notificarDonantesInactivos(): void «@Scheduled 0 0 0 * * *»
- private notificarDonante(donante: PersonaDonante): void
- private mapearTipoMedio(medio: MedioDeContacto): String

CsvImportService
- private BATCH_SIZE: int = 500
- private donanteRepository: PersonaDonanteRepositoryInterface
- private parser: DonanteCsvRowParser
- private factory: DonanteFactory
- public importar(csvStream: InputStream): ImportReport
- private procesarLote(lote: List<DonanteImportDto>, report: ImportReport, lineaInicio: int): void
- private procesarFila(dto: DonanteImportDto, linea: int): ImportFilaCSV

Las dependencias de cada service hacia las interfaces de interfaces.repositories figuran como atributos tipados y no se dibujan como flechas, para no saturar el diagrama. Los métodos «@Scheduled» indican la expresión cron del proceso nocturno.

interfaces

integracion

«interface» NotificacionTransport
- public enviar(solicitud: SolicitudNotificacionMensaje): void
- public nombre(): String

services

«interface» AlgoritmoAsignacionInterface
- public evaluar(donacion: Donacion, entidades: List<EntidadBeneficiaria>): List<ResultadoAsignacion>

«interface» SegmentadorDonacionesServiceInterface
- public segmentar(carga: CargaDonacion): List<Donacion>
- public cargarDonaciones(donaciones: List<Donacion>): void

«interface» DonacionServiceInterface
- public obtenerDonaciones(estado: String, idDonante: UUID, subcategoria: String): List<DonacionResponseDTO>
- public obtenerPorId(id: UUID): DonacionResponseDTO
- public cambiarEstado(id: UUID, dto: CambioEstadoRequestDTO): void
- public modificarBien(id: UUID, dto: BienRequestDTO): void
- public eliminar(id: UUID): void
- public obtenerCandidatos(idDonacion: UUID): CandidatosAsignacionResponseDTO
- public obtenerCandidatosPendientes(): List<CandidatosPendientesResponseDTO>
- public asignar(idDonacion: UUID, dto: AsignacionRequestDTO): void

«interface» PersonaDonanteServiceInterface
- public registrar(dto: PersonaDonanteRequestDTO): PersonaDonanteResponseDTO
- public obtenerDonante(id: UUID): PersonaDonanteResponseDTO
- public obtenerDonantes(estado: String): List<PersonaDonanteResponseDTO>
- public cambiarEstado(id: UUID, dto: EstadoDonanteRequestDTO): void
- public modificarContacto(id: UUID, dto: MedioDeContactoRequestDTO): void
- public modificarRepresentante(id: UUID, dto: RepresentanteRequestDTO): void
- public actualizar(id: UUID, dto: PersonaDonanteUpdateRequestDTO): PersonaDonanteResponseDTO
- public eliminar(id: UUID): void

«interface» EntidadesBeneficiariasServiceInterface
- public guardar(dto: EntidadBeneficiariaRequestDTO): EntidadBeneficiariaResponseDTO
- public obtenerTodas(): List<EntidadBeneficiariaResponseDTO>
- public obtenerPorId(id: UUID): EntidadBeneficiariaResponseDTO
- public actualizar(id: UUID, dto: EntidadBeneficiariaRequestDTO): void
- public eliminarEntidad(id: UUID): void
- public agregarCampaniaAEntidad(idEntidad: UUID, dto: CampaniaRequestDTO): CampaniaResponseDTO
- public agregarNecesidadACampania(entidadId: UUID, campaniaId: UUID, dto: NecesidadRequestDTO): NecesidadResponseDTO
- public obtenerNecesidadesDeCampania(entidadId: UUID, campaniaId: UUID): List<NecesidadResponseDTO>
- public obtenerNecesidad(entidadId: UUID, campaniaId: UUID, necesidadId: UUID): NecesidadResponseDTO
- public actualizarNecesidad(entidadId: UUID, campaniaId: UUID, necesidadId: UUID, dto: NecesidadRequestDTO): NecesidadResponseDTO
- public eliminarNecesidad(entidadId: UUID, campaniaId: UUID, necesidadId: UUID): void

repositories

«interface» DonacionesRepositoryInterface
- public cargarDonaciones(donaciones: List<Donacion>): void
- public guardar(donacion: Donacion): void
- public obtenerTodas(): List<Donacion>
- public obtenerPorEstado(estado: String): List<Donacion>
- public obtenerPorDonante(idDonante: UUID): List<Donacion>
- public obtenerPorId(id: UUID): Donacion
- public eliminar(idDonacion: UUID): void

«interface» EntidadesBeneficiariasRepositoryInterface
- public guardar(entidad: EntidadBeneficiaria): void
- public buscarTodas(): List<EntidadBeneficiaria>
- public obtenerPorId(id: UUID): EntidadBeneficiaria
- public existePorId(id: UUID): boolean
- public eliminar(id: UUID): void

«interface» PersonaDonanteRepositoryInterface
- public obtenerTodosDonantes(): List<PersonaDonante>
- public guardar(donante: PersonaDonante): void
- public modificarRepresentante(idDonante: UUID, representante: Representante): void
- public existePorId(id: UUID): boolean
- public existePorEmail(email: String): boolean
- public obtenerPersona(id: UUID): PersonaDonante
- public obtenerPorEmail(email: String): PersonaDonante
- public obtenerPorEstado(estado: String): List<PersonaDonante>
- public eliminar(id: UUID): void

«interface» CandidatosAsignacionRepositoryInterface
- public guardar(ranking: RankingPrecalculado): void
- public obtenerPorDonacion(idDonacion: UUID): RankingPrecalculado
- public obtenerTodos(): List<RankingPrecalculado>
- public eliminar(idDonacion: UUID): void
- public limpiar(): void

validations

donaciones

DonacionesValidator
- private repositorio: DonacionesRepositoryInterface
- public validarYObtenerDonacion(id: UUID): Donacion
- public validarTieneBienes(donacion: Donacion): void

entidades

EntidadesBeneficiariasValidator
- private repositorio: EntidadesBeneficiariasRepositoryInterface
- public validarExistenciaEntidad(id: UUID): void
- public validarYObtenerEntidad(id: UUID): EntidadBeneficiaria
- public validarYObtenerCampania(entidad: EntidadBeneficiaria, campaniaId: UUID): Campania
- public validarYObtenerNecesidad(campania: Campania, necesidadId: UUID): Necesidad
- public validarFechasCampania(inicio: LocalDate, fin: LocalDate): void

personas

PersonasValidator
- private EMAIL_REGEX: String
- private repositorio: PersonaDonanteRepositoryInterface
- public validarEmail(email: String): void
- public validarExistenciaPersona(id: UUID): void
- public validarYObtenerPersona(id: UUID): PersonaDonante
- public validarEsPersonaJuridica(id: UUID): void
- public validarMedioContacto(medio: MedioDeContacto): void

repositories

DonacionesRepository
- private jpa: DonacionJpaRepository
- public cargarDonaciones(cargaDonaciones: List<Donacion>): void
- public guardar(donacion: Donacion): void
- public obtenerTodas(): List<Donacion>
- public obtenerPorEstado(estado: String): List<Donacion>
- public obtenerPorDonante(idDonante: UUID): List<Donacion>
- public obtenerPorId(id: UUID): Donacion
- public eliminar(idDonacion: UUID): void

EntidadesBeneficiariasRepository
- private jpa: EntidadBeneficiariaJpaRepository
- public guardar(entidad: EntidadBeneficiaria): void
- public buscarTodas(): List<EntidadBeneficiaria>
- public obtenerPorId(id: UUID): EntidadBeneficiaria
- public existePorId(id: UUID): boolean
- public eliminar(id: UUID): void

PersonaDonanteRepository
- private jpa: PersonaDonanteJpaRepository
- public guardar(personaDonante: PersonaDonante): void
- public obtenerPersona(id: UUID): PersonaDonante
- public obtenerTodosDonantes(): List<PersonaDonante>
- public modificarRepresentante(idPersonaJuridica: UUID, representante: Representante): void
- public existePorId(id: UUID): boolean
- public existePorEmail(email: String): boolean
- public obtenerPorEmail(email: String): PersonaDonante
- public obtenerPorEstado(estado: String): List<PersonaDonante>
- public eliminar(id: UUID): void

CandidatosAsignacionRepository
- private almacenamiento: Map<UUID, RankingPrecalculado>
- public guardar(ranking: RankingPrecalculado): void
- public obtenerPorDonacion(idDonacion: UUID): RankingPrecalculado
- public obtenerTodos(): List<RankingPrecalculado>
- public eliminar(idDonacion: UUID): void
- public limpiar(): void

DonacionesRepository, EntidadesBeneficiariasRepository y PersonaDonanteRepository son adapters sobre Spring Data JPA (base donaciones_db). CandidatosAsignacionRepository guarda los rankings en memoria (ConcurrentHashMap): no se persisten.

jpa

«interface» DonacionJpaRepository
- public findByEstadoInAndSubcategoriaNombre(estados: String): List<Donacion>
- public findBySolDonante(idDonante: UUID): List<Donacion>

«interface» EntidadBeneficiariaJpaRepository

«interface» PersonaDonanteJpaRepository
- public findByEmailIgnoreCase(email: String): Optional<PersonaDonante>
- public existsByEmailIgnoreCase(email: String): boolean
- public findByEstadoNombre(estadoNombre: String): List<PersonaDonante>

Spring Data JPA

«external» JpaRepository<Donacion, UUID>

«external» JpaRepository<EntidadBeneficiaria, UUID>

«external» JpaRepository<PersonaDonante, UUID>

clientes

IncentivosClient
- private restClient: RestClient
- public notificarDonacionRegistrada(donanteId: UUID, destinatario: String, medio: String, cantidadBienes: int, categoriasDonadas: List<String>): void
- public notificarDonacionRegistrada(donanteId: UUID, destinatario: String, medio: String, cantidadBienes: int, categoriasDonadas: List<String>, fecha: LocalDateTime, entidadBeneficiaria: String): void
- public notificarDonacionExitosa(donacion: Donacion, destinatario: String, medio: String): void
- public notificarDonacionExitosa(donacion: Donacion, destinatario: String, medio: String, fechaEvento: LocalDateTime): void
- private notificarDonacionExitosa(donanteId: UUID, destinatario: String, medio: String, cantidadBienes: int, categoriasDonadas: List<String>, fecha: LocalDateTime, entidadBeneficiaria: String): void
- private cantidadBienes(donacion: Donacion): int
- private categoriasDonadas(donacion: Donacion): List<String>
- private entidadBeneficiaria(donacion: Donacion): String

«create»

NotificacionClient
- private transporte: NotificacionTransport
- public registrarTransporteAlIniciar(): void «@PostConstruct»
- public enviarNotificacion(destinatario: String, mensaje: String, medio: String): void

«record» ResultadoEnvio
- proveedor: String
- proveedoresDescartados: List<String>
- envios: List<EnvioAsignado>

«record» EnvioAsignado
- idDonacion: UUID
- proveedor: String
- idSeguimiento: String
- fechaAsignacion: LocalDateTime

BrokerLogisticaClient
- private restClient: RestClient
- public solicitarEnvio(proveedor: String, donaciones: List<DonacionEnvio>): ResultadoEnvio

«record» SolicitudEnvio
- proveedor: String
- donaciones: List<DonacionEnvio>

«record» DonacionEnvio
- idDonacion: UUID
- idEntidadBeneficiaria: UUID
- nombreEntidad: String
- direccionEntrega: DireccionEnvio

«record» DireccionEnvio
- calle: String
- numero: int
- localidad: String
- provincia: String
- codigoPostal: String

IncentivosClient → POST /api/incentivos/donantes/donacion y /donacion-exitosa (servicio-incentivos, 8083)
BrokerLogisticaClient → POST /api/broker/envios (servicio-broker-logistica, 8087)

mappers

DonacionMapper
- public toDTO(donacion: Donacion): DonacionResponseDTO
- public toDTOList(donaciones: List<Donacion>): List<DonacionResponseDTO>
- public toCargaDonacion(dto: DonacionRequestDTO): CargaDonacion
- public toBien(dto: BienRequestDTO): Bien
- private toBienesDTO(bienes: List<Bien>): List<BienResponseDTO>
- private toBienDTO(bien: Bien): BienResponseDTO
- private toHistorialDTO(historial: List<CambioEstado>): List<CambioEstadoResponseDTO>

EntidadBeneficiariaMapper
- private dependenciasMapper: PersonaDonanteMapper
- public toModel(dto: EntidadBeneficiariaRequestDTO): EntidadBeneficiaria
- public toDTO(entidad: EntidadBeneficiaria): EntidadBeneficiariaResponseDTO
- public toCampaniaModel(dto: CampaniaRequestDTO, idEntidad: UUID): Campania
- public toCampaniaDTO(campania: Campania): CampaniaResponseDTO
- public toNecesidadModel(dto: NecesidadRequestDTO): Necesidad
- public actualizarNecesidadDesde(necesidad: Necesidad, dto: NecesidadRequestDTO): void
- public toNecesidadDTO(necesidad: Necesidad): NecesidadResponseDTO
- private aplicarCamposComunes(necesidad: Necesidad, dto: NecesidadRequestDTO): void
- private nombreTipo(necesidadDto: Object): String

PersonaDonanteMapper
- public toModel(dto: PersonaDonanteRequestDTO): PersonaDonante
- public toDTO(persona: PersonaDonante): PersonaDonanteResponseDTO
- public toContacto(dto: MedioDeContactoRequestDTO): MedioDeContacto
- public toContactoDTO(c: MedioDeContacto): MedioDeContactoResponseDTO
- public toContactosDTO(contactos: List<MedioDeContacto>): List<MedioDeContactoResponseDTO>
- public toDireccion(dto: DireccionRequestDTO): Direccion
- public toLocalidad(dto: LocalidadRequestDTO): Localidad
- public toDireccionDTO(dir: Direccion): DireccionResponseDTO
- public toLocalidadDTO(localidad: Localidad): LocalidadResponseDTO
- public toRepresentante(dto: RepresentanteRequestDTO): Representante
- public toRepresentantesDto(dtos: List<RepresentanteRequestDTO>): List<Representante>
- public toRepresentantesDTO(representantes: List<Representante>): List<RepresentanteResponseDTO>

importacion

DonanteFactory
- public crearPersona(dto: DonanteImportDto): PersonaDonante

DonanteCsvRowParser
- public parsear(columnas: String[], numeroLinea: int): DonanteImportDto

«create»

dto

«record» DonanteImportDto
- tipoPersona: String
- tipoDoc: String
- documento: String
- nombreORazonSocial: String
- email: String
- telefono: String

ImportReport
- private resultados: List<ImportFilaCSV>
- public agregar(resultado: ImportFilaCSV): void
- public totalCreados(): long
- public totalActualizados(): long
- public getErrores(): List<ImportFilaCSV>
- public tieneErrores(): boolean
- public totalProcesadas(): int

ImportFilaCSV
- private numeroLinea: int
- private estado: EstadoImport
- private email: String
- private detalle: String
- public creado(linea: int, email: String): ImportFilaCSV
- public actualizado(linea: int, email: String): ImportFilaCSV
- public error(linea: int, email: String, motivo: String): ImportFilaCSV
- public esError(): boolean

«enum» EstadoImport
- CREADO
- ACTUALIZADO
- ERROR